

SPU2D Bin Conflicts: Empacotamento Bidimensional em Faixas com Restrições de Conflitos

Ycaro Sales

18 de junho de 2026

Resumo

Este trabalho aborda o Problema de Empacotamento Bidimensional em Faixas com Restrições de Conflitos (*2D Strip/Bin Packing with Conflicts*), uma variante NP-difícil do empacotamento clássico em que pares de itens incompatíveis não podem ser alocados em um mesmo recipiente. O objetivo é empacotar todos os itens retangulares no menor número possível de faixas de dimensões fixas, respeitando a ausência de sobreposição e o grafo de conflitos. Apresentamos uma formulação do problema, uma representação da solução baseada em chaves aleatórias e duas estratégias de resolução: uma heurística construtiva gulosa do tipo *First-Fit-Decreasing* e uma metaheurística *Biased Random-Key Genetic Algorithm* (BRKGA). Os experimentos sobre 18 instâncias mostram que o BRKGA reduz o número total de faixas de 91 para 78 (redução média de aproximadamente 14,3%), superando a heurística gulosa em 11 instâncias e empatando nas demais, sem nunca produzir resultado pior.

Sumário

1	Introdução	3
2	Caracterização do problema	3
2.1	O problema do empacotamento bidimensional em faixa com restrições de descarregamento e de conflitos	3
3	Fundamentação teórica	4
4	Metodologia	4
4.1	Visão geral	4
4.2	Representação da solução	5
4.3	Heurística gulosa (<i>First-Fit-Decreasing</i>)	5
4.4	Metaheurística BRKGA	6
5	Resultados	6
6	Conclusão	7

1 Introdução

A logística ocupa papel central na economia contemporânea: o transporte e o armazenamento de mercadorias representam parcela expressiva dos custos operacionais de empresas de manufatura, varejo e comércio eletrônico. Nesse contexto, problemas de *corte e empacotamento* (*cutting and packing*) surgem sempre que é preciso acomodar um conjunto de objetos dentro de recipientes maiores — caixas em paletes, itens em contêineres, peças em chapas de matéria-prima — de forma a aproveitar ao máximo o espaço disponível. Cada recipiente economizado significa menos viagens, menos área de estoque e menor desperdício de material, com impacto direto sobre custos e sustentabilidade.

Este trabalho trata de uma generalização do empacotamento bidimensional em que, além das restrições geométricas usuais, existe a noção de *conflito* entre itens. Dois itens em conflito não podem compartilhar o mesmo recipiente. Esse tipo de restrição modela situações reais como produtos químicos incompatíveis, mercadorias destinadas a clientes ou rotas distintas, ou itens que, por razões de segurança ou regulação, não podem ser transportados juntos. O acréscimo dos conflitos torna o problema substancialmente mais difícil, pois reduz a liberdade de combinação dos itens e tende a aumentar o número de recipientes necessários.

O objetivo deste trabalho é minimizar o número de faixas (recipientes) utilizadas para empacotar todos os itens de uma instância. Para isso, implementou-se em Rust um *solver* que oferece duas abordagens: uma heurística construtiva gulosa, usada como linha de base, e uma metaheurística evolutiva (BRKGA), descritas na Seção 4. As demais seções estão organizadas da seguinte forma: a Seção 2 caracteriza formalmente o problema; a Seção 3 apresenta a fundamentação teórica; a Seção 4 descreve a metodologia, incluindo a representação da solução e as heurísticas empregadas; e a Seção 5 reporta os resultados experimentais.

2 Caracterização do problema

Entrada: um conjunto de faixas (recipientes) retangulares de largura fixa W e altura fixa H ; um conjunto finito $\mathcal{R} = \{r_1, r_2, \dots, r_n\}$ de n retângulos, representando os itens; e um grafo simples de conflitos $G = (\mathcal{R}, E)$, com $E \subseteq \{\{u, v\} \mid u, v \in \mathcal{R} \wedge u \neq v\}$. Cada item r_i possui largura w_i , altura h_i e classe de cliente c_i .

Objetivo: empacotar todos os itens $r \in \mathcal{R}$ dentro das faixas, minimizando o número de faixas utilizadas.

Restrições:

- Todos os itens devem ser empacotados;
- não são permitidas sobreposições entre itens em uma mesma faixa;
- cada item deve estar inteiramente contido nos limites da faixa que o contém ($0 \leq x$, $0 \leq y$, $x + w_i \leq W$, $y + h_i \leq H$), admitindo-se rotação de 90° ;
- a restrição de conflitos deve ser satisfeita: para toda aresta $\{u, v\} \in E$, os itens u e v não podem ser alocados em uma mesma faixa.

2.1 O problema do empacotamento bidimensional em faixa com restrições de descarregamento e de conflitos

O problema estudado é uma variante do empacotamento bidimensional em faixas (*2D Strip/Bin Packing*). Na formulação clássica, busca-se posicionar retângulos sem sobreposição dentro de recipientes de dimensão fixa, minimizando o número de recipientes. Aqui, a essa estrutura acrescentam-se:

(i) a *restrição de conflitos*, modelada pelo grafo G , segundo a qual itens ligados por uma aresta não podem coexistir no mesmo recipiente; e (ii) a informação de *classe de cliente* c_i de cada item, que dá suporte a uma *restrição de descarregamento* — a ordem em que os itens de diferentes clientes são acomodados pode condicionar a viabilidade prática do descarregamento ao longo de uma rota. Neste trabalho, o foco recai sobre a restrição de conflitos e sobre a minimização do número de faixas; a classe de cliente é representada nos dados de entrada e fica disponível para extensões futuras.

3 Fundamentação teórica

O empacotamento bidimensional pertence à família de problemas de corte e empacotamento e é NP-difícil, generalizando o *Bin Packing* unidimensional. A introdução do grafo de conflitos aproxima o problema de formulações de coloração de grafos: se a geometria fosse ignorada, alocar itens a recipientes respeitando os conflitos equivaleria a colorir G com o menor número de cores, em que cada cor representa um recipiente. Assim, o número cromático de G é um limitante inferior para o número de faixas necessárias, pois itens mutuamente em conflito (uma clique em G) exigem recipientes distintos.

Como o problema é intratável na prática para instâncias de tamanho realista, recorre-se a métodos aproximados. As *heurísticas construtivas* produzem uma solução em uma única passagem, tomando decisões locais (por exemplo, regras de ajuste como *First-Fit* e *Best-Fit*); são rápidas, mas podem ficar presas em soluções de qualidade limitada. As *metaheurísticas*, por sua vez, exploram o espaço de soluções de forma iterativa e estocástica, buscando escapar de ótimos locais. Entre elas, os algoritmos genéticos evoluem uma população de soluções por meio de seleção, recombinação (*crossover*) e mutação.

O **BRKGA** (*Biased Random-Key Genetic Algorithm*) é uma metaheurística genética em que cada solução é codificada por um vetor de chaves aleatórias reais no intervalo $[0, 1]$. Um *decodificador* determinístico, específico do problema, traduz esse vetor em uma solução concreta e calcula seu custo. Essa separação entre representação (genérica) e decodificação (específica) permite reutilizar todo o maquinário evolutivo — seleção, *crossover* e mutação operam apenas sobre as chaves — enquanto o conhecimento do domínio fica concentrado no decodificador.

Para o posicionamento geométrico dos itens dentro de cada recipiente, utiliza-se a técnica de *Espaços Máximos Vazios* (*Empty Maximal Spaces*, EMS): o espaço livre de um recipiente é representado por um conjunto de retângulos vazios maximais; ao inserir um item, os espaços que ele intercepta são recortados em novos retângulos e os espaços contidos em outros são descartados, mantendo apenas os maximais.

4 Metodologia

4.1 Visão geral

O *solver* foi implementado em Rust. Cada instância é lida de um arquivo JSON (formatos `Objects/Items` e `Container/ItemTypes`), construindo-se a estrutura `Instance` com os itens, as dimensões da faixa e o grafo de conflitos indexado pela posição de cada item. Para cada instância são executadas duas estratégias — a heurística gulosa e o BRKGA — e as soluções resultantes são serializadas em JSON, registrando o número de faixas utilizadas, os itens não alocados, o tempo de execução e a posição de cada item.

4.2 Representação da solução

A solução de uma instância é o conjunto de faixas, cada uma contendo as posições (origem, dimensões e orientação) dos itens nela alocados. O componente geométrico central é o **Container**, que mantém o conjunto de Espaços Máximos Vazios (EMS) e as colocações já realizadas. A inserção segue a regra *bottom-left*: dentre os espaços livres que comportam o item, escolhe-se o de menor origem (primeiro em y , depois em x); a checagem de conflito rejeita o item caso algum vizinho seu em G já esteja na faixa.

No BRKGA, a solução é codificada por um vetor de $2n$ chaves aleatórias em $[0, 1]$, interpretado da seguinte forma:

- as primeiras n chaves definem a **ordem de inserção** dos itens: os itens são ordenados de forma crescente pela sua chave;
- as últimas n chaves definem a **orientação** de cada item: uma chave maior ou igual a um limiar $(0,5)$ indica que o item é rotacionado em 90° ; caso contrário, mantém a orientação original.

O decodificador percorre os itens na ordem dada e os insere por *First-Fit* com orientação fixada, abrindo uma nova faixa quando nenhuma das abertas aceita o item; um item que não cabe nem em uma faixa vazia na sua orientação forçada é marcado como não alocado.

4.3 Heurística gulosa (*First-Fit-Decreasing*)

A heurística gulosa serve de linha de base. Os itens são ordenados por área decrescente e inseridos, um a um, por *First-Fit* nas faixas abertas; quando nenhuma faixa aberta aceita o item, abre-se uma nova faixa. A inserção é *ciente de conflitos* e tenta automaticamente as duas orientações do item. O Algoritmo 1 resume o procedimento.

Algoritmo 1: Heurística gulosa First-Fit-Decreasing

Input: instância com itens $\mathcal{R}$, faixa $W \times H$, grafo G
Output: conjunto de faixas B

```

1  $B \leftarrow \emptyset$ ;
2 ordene  $\mathcal{R}$  por área decrescente;
3 foreach item  $r \in \mathcal{R}$  do
4    $alocado \leftarrow falso$ ;
5   foreach faixa  $b \in B$  do
6     if  $b$  aceita  $r$  (sem conflito e com espaço) then
7       insira  $r$  em  $b$  (regra bottom-left, auto-rotação);
8        $alocado \leftarrow verdadeiro$ ; break;
9     end
10  end
11  if  $\neg alocado$  then
12    abra nova faixa  $b'$  e insira  $r$  em  $b'$ ;
13     $B \leftarrow B \cup \{b'\}$ ;
14  end
15 end
16 return  $B$ ;

```

4.4 Metaheurística BRKGA

A metaheurística evolui uma população de vetores de chaves aleatórias. A *função de aptidão* (*fitness*) de um cromossomo é obtida decodificando-o e calculando

$$f = \text{faixas_usadas} + \text{itens_nao_alocados} \cdot (n + 1),$$

a ser *minimizada*. A penalidade $(n + 1)$ por item não alocado garante que qualquer solução inviável seja sempre pior do que qualquer solução viável (que usa, no máximo, n faixas), de modo que a busca prioriza a viabilidade. O processo evolutivo, ilustrado no Algoritmo 2, foi configurado com população de 100 indivíduos, critério de parada de 200 gerações sem melhora, seleção por torneio, *crossover* uniforme e mutação de gene único.

Algoritmo 2: BRKGA para o empacotamento com conflitos

Input: instância, tamanho de população P , máximo de gerações estagnadas S
Output: melhor empacotamento encontrado

```

1  inicialize população de  $P$  vetores de  $2n$  chaves em  $[0, 1]$ ;
2  repeat
3      foreach cromossomo do
4          decodifique (ordem + orientação) e empacote por First-Fit;
5          avalie  $f = \text{faixas} + \text{nao\_alocados} \cdot (n + 1)$ ;
6      end
7      selecione por torneio; aplique crossover uniforme e mutação;
8      forme a nova geração;
9  until  $S$  gerações sem melhora;
10 return decodificação do melhor cromossomo;
```

5 Resultados

As duas estratégias foram avaliadas sobre 18 instâncias derivadas de conjuntos de teste de empacotamento bidimensional, às quais foram adicionados conflitos com diferentes densidades (sufixos c0.1, c0.3 e c0.5, indicando a probabilidade de conflito entre pares de itens). A Tabela 1 compara o número de faixas utilizadas pela heurística gulosa e pelo BRKGA.

Considerando o conjunto completo de 18 instâncias, a heurística gulosa utilizou 91 faixas no total, enquanto o BRKGA utilizou 78 — uma redução média de aproximadamente 14,3%. O BRKGA produziu soluções estritamente melhores em 11 instâncias, empatou em 7 e em nenhum caso foi pior do que a gulosa. Quanto ao tempo de execução, a heurística gulosa é praticamente instantânea ($< 0,01$ s por instância), ao passo que o BRKGA, por seu caráter iterativo, gasta tipicamente entre 0,07 s e 0,13 s por instância — um custo ainda baixo, compatível com seu ganho de qualidade.

Observa-se também que o aumento da densidade de conflitos (de c0.1 para c0.5) eleva consistentemente o número de faixas necessárias, o que é esperado: quanto mais arestas no grafo de conflitos, menos itens podem compartilhar um mesmo recipiente. Nesses cenários mais restritos é justamente onde o BRKGA mais se distancia da heurística gulosa, evidenciando o valor da busca metaheurística quando as restrições de conflito se intensificam.

Tabela 1: Número de faixas utilizadas: gulosa vs. BRKGA.

Instância	n	Gulosa	BRKGA
BPPSP-n100m18b2d2_Hard_c0.1	18	3	3
BPPSP-n100m18b2d2_Hard_c0.5	18	6	5
CJCM-E00N23_HardInfeasible_c0.3	23	6	5
CJCM-E00N23_HardInfeasible_c0.5	23	8	6
CJCM-E02F20_ManyCalls_c0.3	20	5	4
CJCM-E02N20_Hard_c0.3	20	6	4
CJCM-E02N20_Hard_c0.5	20	7	6
CJCM-E03X18_HardFeasible_c0.1	18	3	2
CJCM-E03X18_HardFeasible_c0.5	18	7	6
Total (18 instâncias)		91	78

6 Conclusão

Este trabalho apresentou uma formulação e duas estratégias de resolução para o empacotamento bidimensional em faixas com restrições de conflitos. A representação por chaves aleatórias do BRKGA, combinada a um decodificador *First-Fit* ciente de conflitos e ao gerenciamento de espaços livres via EMS, mostrou-se eficaz: o BRKGA reduziu o número total de faixas em relação à heurística gulosa sem nunca piorar nenhuma instância. Como trabalhos futuros, destacam-se a incorporação efetiva da restrição de descarregamento por classe de cliente, a avaliação sobre um conjunto maior de instâncias e o ajuste sistemático dos parâmetros da metaheurística.

Referências Bibliográficas

Referências

- [1] Gonçalves, J. F.; Resende, M. G. C. (2011). *Biased random-key genetic algorithms for combinatorial optimization*. Journal of Heuristics, 17(5), 487–525.
- [2] Lodi, A.; Martello, S.; Monaci, M. (2002). *Two-dimensional packing problems: A survey*. European Journal of Operational Research, 141(2), 241–252.
- [3] Wäscher, G.; Haußner, H.; Schumann, H. (2007). *An improved typology of cutting and packing problems*. European Journal of Operational Research, 183(3), 1109–1130.